

Micetro External Discovery

Version 0.9.23

OVERVIEW	3
CURRENT TESTED PLATFORMS	3
COMPONENTS	3
CONFIGURATION	3
setup.json	3
url	4
validateCert	4
skipCIDRs	4
skipIPs & skipNetworks	4
vrfOverlap	4
logFileName	4
logFileSizeInBytes	5
logFileBackupCount	5
encryptAuth	5
fernetUser	5
logLevel	5
timeout	5
snmpRetries	5
maxPIDAge	5
L2Discovery [true/false]	5
alertOnDeviceNotSeenInterval [int(hours)]	5
alertOnDeviceNotSeenSyslog [IP/FQDN:udp/tcp:port]	6
Palo Alto specific	6
paloNAT	6
pNATProperty	6
profiles.json	7
Sample profiles.json Config	8
Examples –	9
SNMP version 2c discovery against a router using the standard Micetro mibs.	9
As above but with an additional 2 routers	10
Disabled profile. The script will skip this profile during execution	10
Overriding the default oid used under IP-MIB for ipNetToMediaPhysAddress from 1.3.6.1.2.1.4.22.1.2 → 1.3.6.1.2.1.4.35.1.2 (Cisco FirePower)	10
PANOS API profile for two PaloAlto's using API keys	11
Cisco Nexus API credentials for one router	11
SNMPv3 example using MD5/DES where credentials have been automatically encrypted after the first run (note EEEEE# prefix in front of the encrypted data)	11
L2 Discovery. Uses the BRIDGE-MIB against compatible switches to find port details	12
Checking the JSON format files	12
INSTALLATION	13
Copy the script and config files to the central script folder	13
Disable the built-in discovery	13
Preparing Micetro for externalDiscovery	14
Activate the ScriptRunner account	14
Enable discovery info	14
Activating externalDiscovery	15
Configuring Event Hooks	15
Testing	16
Checking the log	17
APPENDIX	18
Common issues	18
Backing up the SQLite database	18

Using snmpwalk to test different oids	19
FORTI-OS	20
Creating binary executables from python code using pyinstaller	23
API Commands used to retrieve data	23
Cisco Nexus	23
Palo Alto	24
Create a fake API for testing	24

Overview

externalDiscovery is a Python based script which can be used for network discovery in Micetro. It extends Micetro network discovery capabilities using SNMP and vendor APIs where the built-in discovery engine is insufficient.

Examples of this include –

- Where SNMP isn't available/working against PaloAlto but the PANOS API is available
- Where SNMP isn't available/working against FortiGate but the FORTI API is available
- Where support for VRFs is required (currently tested against Cisco IOS and Nexus)
- Where the customer has configured loopbacks within ranges causing unwanted /32, /31 networks to appear in the GUI during standard SNMP Discovery
- Where a non-standard SNMP oid could be used to retrieve ARP and Subnet data
- Discovery filtering and subnet suppression
- L2 discovery using the SNMP BRIDGE MIB

Current tested platforms

- PANOS
- Cisco Nexus
- Cisco IOS
- VyOS
- Netgear
- FortiOS

Components

externalDiscovery.py	Python Source code. Not recommended to be used in Customer environments
externalDiscovery	x64 Linux binary created using pyinstaller
externalDiscovery.exe	x64 Windows binary created using pyinstaller
setup.json	JSON format configuration file. Needs to be tailored for the target environment
profiles.json	JSON format router profile file
Micetro_External_discovery.pdf	This document

Configuration

The externalDiscovery script references 2 configuration files

- setup.json - Configuration data specific to the Micetro environment
- profiles.json - Configuration data specific to routers/switches for discovery

setup.json

setup.json contains all the setup information unique to the Micetro install in JSON format.

Example setup.json config

```
{
  "url": "http[s]://server_name|IP_address/mmws/api/command",
  "validateCert": false,
  "skipCIDRs": ["31", "32"],
  "skipIPs": ["127.0.0.1", "0.0.0.1", "254.0.0.1"],
  "vrfOverlap": false,
  "skipNetworks": ["127.0.0.1", "10.24.32.0"],
  "logFileName": "snmp.log",
  "logFileSizeInBytes": 10485760,
  "logFileBackupCount": 5,
  "encryptAuth": true,
```

```

"fernetUser" : "Fernet",
"logLevel" : 10,
"timeout" : 5,
"snmpRetries" : 3
}

```

url

The location where the soap API can be found on the Micetro webserver

Eg:

- <http://micetro.micetrolab.com/mmws/api/command>
- <https://micetro.micetrolab.com/mmws/api/command>
- <http://192.168.12.2/mmws/api/command>

validateCert

If using https for connections to the SOAP API do we validate the SSL certificate?

- true - If using a valid PKI issued certificate
- false - If using self-signed

skipCIDRs

A list of CIDR subnet masks which if found matching should not be created as subnets within Micetro

Eg:

- [32]
- [31,32]

This prevents the issue seen in the built in SNMP discovery where if a loopback is configured within a subnet then the /31/32 gets created as a sub-network breaking DHCP usage stats.

RANGE	TYPE	UTILIZATION	AUTHORITY	FAILOVE...
0.0.0.0/0	CONTAINER	---	---	---
10.20.0.0/16	CONTAINER	---	---	---
10.20.200.0/24	RANGE	---	---	---
10.20.200.3/32	RANGE	0%	---	---

Figure 1 Example of /32 loopback discovery creating unwanted range

Any ARP entries discovered related to skipCIDRs get added to the parent network. For example the MAC address relating to 10.20.100.3 will get added to 10.20.200.0/24

skipIPs & skipNetworks

A list of IP addresses / Networks that if discovered won't be added to Micetro

vrfOverlap

If VRFs are detected are these within the current namespace or are they overlapping

- true - overlapping. We need to assign these to unique namespaces (not currently supported)
- false - all VRFs are using unique addressing

logFileName

The name of the log file to create relative to the directory containing central (one level above scripts)

logFileSizeInBytes

Max size before rotation

logFileBackupCount

Number of copies to retain

encryptAuth

If using SNMP3 or API credentials do we want to encrypt these?

- true - A Fernet key (<https://cryptography.io/en/latest/fernet/>) will be stored in the fernetUser (specified below) account within Micetro to be used to encrypt/decrypt data. Any clear text authentication data within profiles.json will be encrypted with this
- false - No encryption required

fernetUser

The Micetro user account which will be used to store the Fernet Key

logLevel

The numeric value associated with the required log level. See table below

Level	Numeric value	What it means / When to use it
logging. NOTSET	0	When set on a logger, indicates that ancestor loggers are to be consulted to determine the effective level. If that still resolves to NOTSET, then all events are logged. When set on a handler, all events are handled
logging. DEBUG	10	Detailed information, typically only of interest to a developer trying to diagnose a problem
logging. INFO	20	Confirmation that things are working as expected
logging. WARNING	30	An indication that something unexpected happened, or that a problem might occur in the near future (e.g. 'disk space low'). The software is still working as expected
logging. ERROR	40	Due to a more serious problem, the software has not been able to perform some function
logging. CRITICAL	50	A serious error, indicating that the program itself may be unable to continue running

timeout

Timeout in seconds when waiting for a non-responsive host

snmpRetries

The number of times to re-try a connection before failing

maxPIDAge

If the timestamp on the pidfile is not updated in n minutes we'll assume the script execution was interrupted and will therefore delete it

L2Discovery [true/false]

If present in setup.json and set to 'true' additional IPAddress related custom properties will be created to track port/vlan details for L2 devices. This also requires L2 device config under profiles and SNMP BRIDGE-MIB support.

alertOnDeviceNotSeenInterval [int(hours)]

If present in setup.json. An integer value in hours for the script to check when a device was last successfully polled. Also creates an additional custom property 'monitor': Yes/No set against devices. If configured the script will check when devices were last seen at the end of the run, If not

seen within the configured interval an error alert will be sent to the log file. This can be disabled by setting monitor to No within the Micetro GUI

alertOnDeviceNotSeenSyslog [IP/FQDN:udp/tcp:port]

Works in conjunction with alertOnDeviceNotSeenInterval. If set will additionally send an alert to the target syslog server. Eg.

```
"alertOnDeviceNotSeenSyslog": "192.168.1.64:udp:514"
```

Will send a syslog alert to server 192.168.1.64 using udp on the standard syslog port 514

```
"alertOnDeviceNotSeenSyslog": "monitor.internal.lab:tcp:518"
```

Will send a syslog alert to server monitor.internal.lab using tcp on the non-standard port 518

Date	Time	Host Name	Program	Messages
04/15/2026	15:17:41	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device MikroTik has not been seen for 168 hours. Please investigate
04/15/2026	15:17:41	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device vyos has not been seen for 1080 hours. Please investigate
04/14/2026	17:10:32	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device MikroTik has not been seen for 145 hours. Please investigate
04/14/2026	17:10:32	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device vyos has not been seen for 1058 hours. Please investigate
04/14/2026	17:07:35	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device MikroTik has not been seen for 145 hours. Please investigate
04/14/2026	17:07:35	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device vyos has not been seen for 1058 hours. Please investigate
04/14/2026	17:07:01	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device MikroTik has not been seen for 145 hours. Please investigate
04/14/2026	17:07:01	micetro.demo.lab	INFO	externalDiscovery.py - Alert from Micetro externalDiscovery. Device vyos has not been seen for 1058 hours. Please investigate

Table 1 Example syslog alert received by server

Palo Alto specific

paloNAT

If set to true, runs the PaloAlto show running nat-policy-address CLI command over the API
Eg.

```
"paloNAT" : true,
```

pNATProperty

The IP Address level Custom Property to create/use to store the destination NAT address
Eg.

```
"pNATProperty" : "externalNAT"
```

Results in- *Timestamp * Router_Name * Nat Policy Name* being displayed under externalNAT

← BACK TO LIST ✎ PROPERTIES ✕ CLEAR ▼ ACTION 🔍 Quick filter 🔊 🔄					
ADDRESS	STATE	LAST KNOWN MAC ADDRESS	EXTERNALNAT	LAST SEEN	
○ 195.200.201.10	ASSIGNED	---	2024-08-23 11:37:34 * PA-VM * test4	---	
○ 195.200.201.90	ASSIGNED	---	2024-08-23 11:37:34 * PA-VM * test3	---	
○ 195.200.201.195	ASSIGNED	---	2024-08-23 11:37:34 * PA-VM * test2	---	
○ 195.200.201.12	ASSIGNED	---	2024-08-23 11:37:34 * PA-VM * MB-Test	---	

profiles.json

profiles.json is where all the local switch/router SNMP/API details are stored.

profiles	Array containing multiple elements			
enabled	true/false			
profileName	Unique string describing the profile			
discoverytype	SNMP	NX-OS	PAN-OS	FORTI-OS
deviceType	[null] L2 L3 If set to L2 enables BRIDGE-MIB discovery			
version	1 2c 3			
communityString	RO SNMPv1/2c community string			
user	SNMPv3 username			
authProtocol	Derived from the Python UsmUserData class- https://docs.alexandros.com/pysnmp/v7.1/docs/api-reference#pysnmp.hlapi.v3arch.asyncio.UsmUserData usmNoAuthProtocol usmHMACMD5AuthProtocol usmHMACSHAAuthProtocol usmHMAC128SHA224AuthProtocol usmHMAC192SHA256AuthProtocol usmHMAC256SHA384AuthProtocol usmHMAC384SHA512AuthProtocol			
authpass	SNMPv3 password			
encrProtocol	Derived from the Python USMUserData class- https://docs.alexandros.com/pysnmp/v7.1/docs/api-reference#pysnmp.hlapi.v3arch.asyncio.UsmUserData usmNoPrivProtocol usmDESPrivProtocol usm3DESEDEPrivProtocol usmAesCfb128Protocol usmAesCfb192Protocol usmAesCfb256Protocol			
encrKey	SNMPv3 encryption key			
apiUser		API User		
apiKey			API key	API key
apiPassword		API Password		
oids	SNMPv1/2c/3 alternatives to the Micetro defaults #Default OIDs used by Micetro #IP-MIB 'ipNetToMediaPhysAddress': '1.3.6.1.2.1.4.22.1.2', 'ipAdEntAddr': '1.3.6.1.2.1.4.20.1.1', 'ipAdEntIfIndex': '1.3.6.1.2.1.4.20.1.2', 'ipAdEntNetMask': '1.3.6.1.2.1.4.20.1.3', 'ifDescr': '1.3.6.1.2.1.2.2.1.2', 'ifAlias': '1.3.6.1.2.1.31.1.1.1.18', #ENTITY-MIB 'entPhysicalSerialNum': '1.3.6.1.2.1.47.1.1.1.1.11', #SNMPv2-MIB 'sysName': '1.3.6.1.2.1.1.5', #CISCO-VLAN-IFTABLE-RELATIONSHIP-MIB 'cviRoutedVlanIfIndex': '1.3.6.1.4.1.9.9.128.1.1.1.1.3', #CISCO-VRF-MIB 'cvVrfInterfaceType': '1.3.6.1.4.1.9.9.711.1.2.1.1.2', 'cvVrfName': '1.3.6.1.4.1.9.9.711.1.1.1.1.2'			
routers	List of IPs	List of IPs	URL including target port	URL including target port

Sample profiles.json Config

```
{
  "profiles": [
    {
      "enabled": false,
      "profileName": "ciscoFirePower",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "oids": {
        "ipNetToMediaPhysAddress": "1.3.6.1.2.1.4.35.1.4"
      },
      "routers": [
        "192.168.45.147"
      ]
    },
    {
      "enabled": true,
      "profileName": "netgear3",
      "discoveryType": "SNMP",
      "version": "3",
      "user": "v3user",
      "authProtocol": "usmHMACMD5AuthProtocol",
      "authpass": "v3userPass",
      "encrProtocol": "usmDESPrivProtocol",
      "encrKey": "12345678",
      "routers": [
        "10.50.0.9"
      ]
    },
    {
      "enabled": true,
      "profileName": "PaloAlto",
      "discoveryType": "PAN-OS",
      "routers": [
        "https://10.50.0.120:443:LUFRT0xWnl0ZzBVYnU2VzhiWHZpbW41ajk1MzkZytpUUF0S3FKZQ==",
        "https://10.50.0.100:443:LUFYYT0xWnl0ZzBVYnU2VzsdhshdgsdHSH023232323dUF0S3FKZQ=="
      ]
    },
    {
      "enabled": true,
      "profileName": "FortiGate",
      "discoveryType": "FORTI-OS",
      "routers": [
        "https://10.50.0.120:443:LUFRT0xWnl0ZzBVYnU2VzhiWHZpbW41ajk1MzkZytpUUF0S3FKZQ==",
        "https://10.50.0.100:443:LUFYYT0xWnl0ZzBVYnU2VzsdhshdgsdHSH023232323dUF0S3FKZQ=="
      ]
    },
    {
      "enabled": true,
      "profileName": "Nexus",
      "discoveryType": "NX-OS",
      "apiUser": "micetro",
      "apiPassword": "micetro123",
      "routers": [
        "10.50.0.120"
      ]
    }
  ]
}
```



```

    ]
  },
  {
    "enabled": true,
    "profileName": "netgear2",
    "discoveryType": "SNMP",
    "version": "2c",
    "communityString": "routers",
    "routers": [
      "10.50.0.4",
      "10.50.0.5"
    ]
  },
  {
    "enabled": true,
    "profileName": "vyos",
    "discoveryType": "SNMP",
    "version": "2c",
    "communityString": "routers",
    "routers": [
      "10.50.0.254",
      "10.50.0.252"
    ]
  }
]
}

```

Ok. I'll be honest there's a trade-off here between flexibility and complexity. Configuring profiles.json can be fairly complex however there is great flexibility here and an ability to tailor the discovery to the your environment rather than just using the standard SNMP mibs used by the default discovery engine. Rather than breaking everything down like was done for setup.json I'll list some specific examples which can then be tweaked accordingly

Examples –

SNMP version 2c discovery against a router using the standard Micetro mibs.

Router IP: 192.168.1.1

SNMP Community: public

Profile: mySwitch # Only used internally within the script

```

{
  "profiles": [
    {
      "enabled": true,
      "profileName": "mySwitch",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "routers": [
        "192.168.1.1"
      ]
    }
  ]
}

```

As above but with an additional 2 routers

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "mySwitch",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "routers": [
        "192.168.1.1",
        "192.168.1.10",
        "192.168.10.1"
      ]
    }
  ]
}
```

Disabled profile. The script will skip this profile during execution

```
{
  "profiles": [
    {
      "enabled": false,
      "profileName": "mySwitch",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "routers": [ "192.168.1.1" ]
    }
  ]
}
```

Overriding the default oid used under IP-MIB for ipNetToMediaPhysAddress from 1.3.6.1.2.1.4.22.1.2 → 1.3.6.1.2.1.4.35.1.2 (Cisco FirePower)

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "ciscoFirePower",
      "discoveryType": "SNMP",
      "version": "2c",
      "communityString": "public",
      "oids": {
        "ipNetToMediaPhysAddress": "1.3.6.1.2.1.4.35.1.4",
        "ipAdEntAddr": "1.3.6.1.2.1.4.20.1.1",
        "ipAdEntIfIndex": "1.3.6.1.2.1.4.20.1.2",
        "ipAdEntNetMask": "1.3.6.1.2.1.4.20.1.3",
        "entPhysicalSerialNum": "1.3.6.1.2.1.47.1.1.1.11",
        "sysName": "1.3.6.1.2.1.1.5"
      },
      "routers": [
        "192.168.45.147"
      ]
    }
  ]
}
```

PANOS API profile for two PaloAlto's using API keys

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "PaloAlto",
      "discoveryType": "PAN-OS",
      "routers": [
        "https://10.50.0.120:443:LUFRT0xWnl0ZzBVYnU2VzhiWHZpbW41ajk1MzkZytpUUF0S3FKZQ==",
        "https://10.50.0.100:443:LUFYYT0xWnl0ZzBVYnU2VzsdhshdgsdHSH02323223dUF0S3FKZQ=="
      ]
    }
  ]
}
```

Cisco Nexus API credentials for one router

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "Nexus",
      "discoveryType": "NX-OS",
      "apiUser": "micetro",
      "apiPassword": "micetro123",
      "routers": [
        "10.10.10.6"
      ]
    }
  ]
}
```

SNMPv3 example using MD5/DES where credentials have been automatically encrypted after the first run (note EEEEE# prefix in front of the encrypted data)

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "netgear3",
      "discoveryType": "SNMP",
      "version": "3",
      "user": "v3user",
      "authProtocol": "usmHMACMD5AuthProtocol",
      "authpass": "EEEEEE#gAAAAABmoi-Lb8sjRzKfFuiXU4X51pP7Q33_CYgaLcuuPAJzjUkPvG00hb7BvbFT8ubH1ReX7KAhobcK4qlwbE1k664Wom3ug==",
      "encrProtocol": "usmDESPrivProtocol",
      "encrKey": "EEEEEE#gAAAAABmoi-Lo0KpA51Y4zcv_GeRisUUVF2BLyf162D0RGbbLhPzxaLZgIrC3YazzTtbYRvkgdsyLt4V0SEPUaYgTeP7Kvp6BA==",
      "routers": [
        "10.50.0.5"
      ]
    }
  ]
}
```

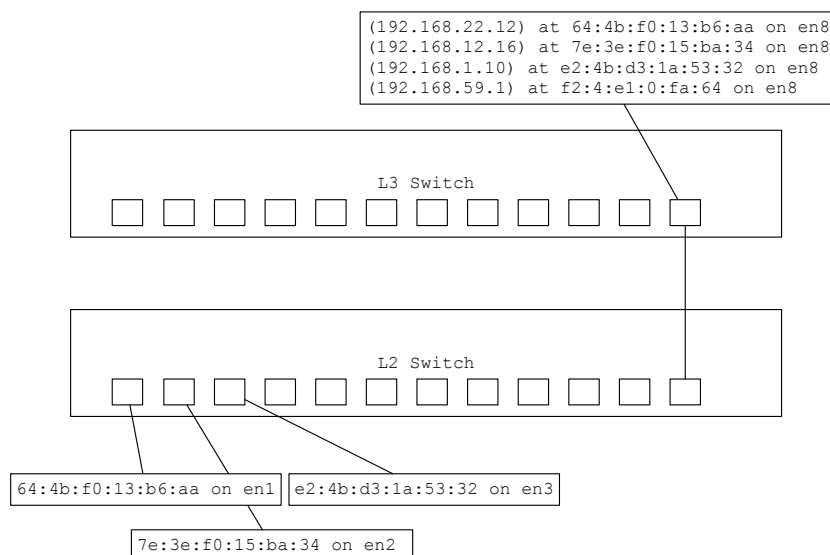
L2 Discovery. Uses the BRIDGE-MIB against compatible switches to find port details

```
{
  "profiles": [
    {
      "enabled": true,
      "profileName": "bridge",
      "discoveryType": "SNMP",
      "deviceType": "L2",
      "version": "2c",
      "communityString": "routers",
      "routers": [
        "10.50.0.8"
      ]
    }
  ]
}
```

If an L2 device is defined then the externalDiscovery script will create and populate the L2VLAN, L2device and L2interface network level custom properties where possible

Address	State	Last known MAC address	Last seen	Device	Interface	L2VLAN	L2device	L2interface
192.168.12.183	Assigned	80:69:1A:16:EB:4C	2026-03-02 15:06:...	---	---	1	NG-bridge	GigabitEthernet8
192.168.12.171	Assigned	A0:CE:C8:D3:2F:7D	2026-03-24 17:06:...	---	---	1	NG-bridge	GigabitEthernet3

Note: The script uses the MAC/IP address discovered against upstream L3 devices in order to calculate which physical port is in use by the L2 device associated with the IP. As per the diagram below where multiple MAC addresses are seen against an individual port these are assumed as learned from a downstream device. Where only one MAC is seen against an L2 device the script uses the info captured at the L3 layer in order to associate the port details with an IP. It is therefore recommended that upstream L3 devices be listed first in profiles.json.



Checking the JSON format files

If you have python installed you can use json.tool as below-

```
python -m json.tool profiles.json
```

Alternatively, something like Visual Studio Code does a good job here

Installation

Copy the script and config files to the central script folder

Important

1. setup.json needs to be tailored for the micetro environment
2. profiles.json needs to be tailored for the switch/router environment

The externalDiscovery[.exe] executable together with the profiles.json and setup.json need to be copied to the scripts folder under the Miceto Central default directory

Eg.

- default location for Windows: C:\ProgramData\Men and Mice\Central\scripts
- default location for Linux: /var/mmsuite/mmcentral/scripts

It's recommended that setup.json and profiles.json are updated prior to transfer and checked to make sure the json is valid prior to transfer (visual studio code does a good job at this).

Disable the built-in discovery

We should either use the Micetro built in discovery or the externalDiscovery script. Using both will likely result in clashes and a bit of a mess in the GUI. Unless you can be sure that routers/switches are only added to one or the other...

- Navigate to ADMIN | CONFIGURATION | SNMP PROFILES and check that there are no existing profiles, or if there are that no routers have been assigned

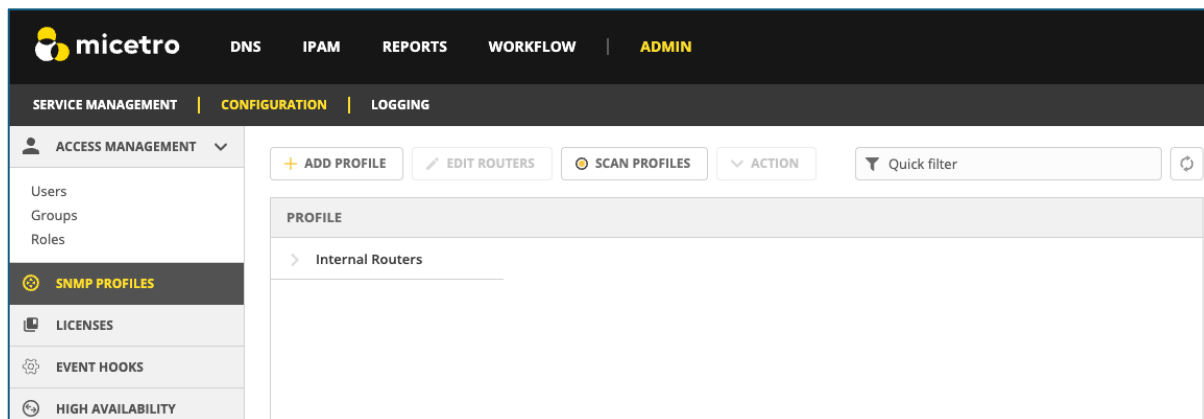


Figure 2 Default Discovery Profiles

Preparing Micetro for externalDiscovery

Before we can run the externalDiscovery script we need a few things setup in Micetro

Activate the ScriptRunner account

As you would for a normal user create an account in Micetro called ScriptRunner and assign this to the administrator role.

The image shows two screenshots of the 'CREATE USER' dialog in Micetro. The left screenshot shows the 'GENERAL' tab with fields for Username (ScriptRunner), Full name, Description, Authentication type (Micetro), Email address, Password, and Confirm password. The right screenshot shows the 'ROLES' tab with a list of roles where 'Administrators (built-in)' is selected.

Although you add ScriptRunner in the normal way, this account is a special account in Micetro in that when used with scripts for security Micetro generates a unique password at each run.

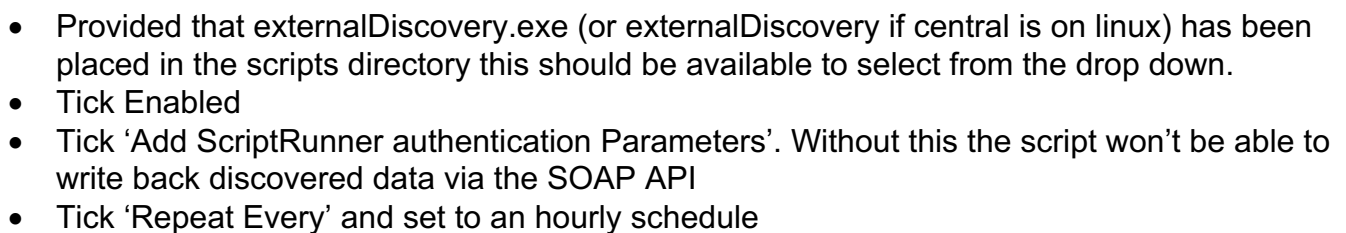
Enable discovery info

Check that 'Always show discovery info' is ticked under System Settings

The image shows a screenshot of the Micetro 'SYSTEM SETTINGS' page. The 'IPAM' section is expanded, showing options for 'Always show discovery info' which is checked.

Configuring Event Hooks

- Navigate to ADMIN | CONFIGURATION | EVENT HOOKS then click [+ ADD] under scheduled events



Scheduled events ?					
Script name	Enabled	Last run 🕒	Next run 🕒	Run count	Repeat
scripts/externalDiscovery.ex...	Yes	2024-07-25 14:09:10	2024-07-25 14:34:00	1	1 hours

Testing

- Select 'Run now'

Scheduled events

Script name	Enabled	Last run	Next run	Run count	Repeat	
scripts/externalDiscovery.exe...	Yes	2024-07-25 14:09:10	2024-07-25 14:34:00	1	1 hours	...

+ ADD

Edit

Run now

Remove

- Select 'YES'

RUN SCHEDULED EVENT NOW

Are you sure you want to run "scripts/externalDiscovery.exe \$u \$p" now?

NO

YES

- This should complete without error. As below

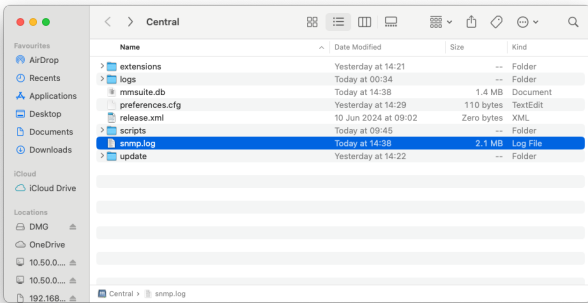
RUNNING SCRIPT

EXIT CODE: 0

OUTPUT:

Checking the log

Note. The amount of detail in the logs will vary depending on the debug level set in setup.json
The default location for the log is in the directory containing central. The logname and location can be adjusted relative to the central directory, again in setup.json



Eg. in DEBUG mode-

```
2024-07-25 11:57:19,400 - Processing:10.50.0.5 in profile netgear2
2024-07-25 11:57:19,400 - Using SNMP version: 2c
2024-07-25 11:57:19,598 - Running getRoutersInterfacesWithSNMP
2024-07-25 11:57:19,588 - Processing interfaces
2024-07-25 11:57:19,588 - Using SNMP version: 2c
2024-07-25 11:57:19,760 - walkresults [{"SNMPv2-SMI::mib-2.4.20.1.2.10.50.0.5 = 66", 'SNMPv2-SMI::mib-2.4.20.1.2.10.50.0.5 = 68', 'SNMPv2-SMI::mib-2.4.20.1.2.192.168.12.5 = 67', 'SNMPv2-SMI::mib-2.4.20.1.2.192.168.88.5 = 53'}]
2024-07-25 11:57:19,760 - Processing interface names
2024-07-25 11:57:19,760 - Using SNMP version: 2c
2024-07-25 11:57:20,072 - walkresults [{"SNMPv2-SMI::mib-2.2.2.1.2.1 = Slot: 0 Port: 1 Gigabit - Level", 'SNMPv2-SMI::mib-2.2.2.1.2.2 = Slot: 0 Port: 2 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.3 = Slot: 0 Port: 3 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.4 = Slot: 0 Port: 4 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.5 = Slot: 0 Port: 5 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.6 = Slot: 0 Port: 6 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.7 = Slot: 0 Port: 7 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.8 = Slot: 0 Port: 8 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.9 = Slot: 0 Port: 9 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.10 = Slot: 0 Port: 10 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.11 = Slot: 0 Port: 11 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.12 = Slot: 0 Port: 12 Gigabit - Level', 'SNMPv2-SMI::mib-2.2.2.1.2.53 = CPU Interface for Slot: 5 Port: 1', 'SNMPv2-SMI::mib-2.2.2.1.2.54 = lag 1', 'SNMPv2-SMI::mib-2.2.2.1.2.55 = lag 2', 'SNMPv2-SMI::mib-2.2.2.1.2.56 = lag 3', 'SNMPv2-SMI::mib-2.2.2.1.2.57 = lag 4', 'SNMPv2-SMI::mib-2.2.2.1.2.58 = lag 5', 'SNMPv2-SMI::mib-2.2.2.1.2.59 = lag 6', 'SNMPv2-SMI::mib-2.2.2.1.2.60 = lag 7', 'SNMPv2-SMI::mib-2.2.2.1.2.61 = lag 8', 'SNMPv2-SMI::mib-2.2.2.1.2.62 = lag 9', 'SNMPv2-SMI::mib-2.2.2.1.2.63 = lag 10', 'SNMPv2-SMI::mib-2.2.2.1.2.64 = lag 11', 'SNMPv2-SMI::mib-2.2.2.1.2.65 = lag 12', 'SNMPv2-SMI::mib-2.2.2.1.2.66 = vlan 1040', 'SNMPv2-SMI::mib-2.2.2.1.2.67 = vlan 1012', 'SNMPv2-SMI::mib-2.2.2.1.2.68 = vlan 1050'}]
2024-07-25 11:57:20,072 - Processing interface descriptions
2024-07-25 11:57:20,072 - Using SNMP version: 2c
2024-07-25 11:57:20,353 - Getting Network data from 10.50.0.5
2024-07-25 11:57:20,353 - Using SNMP version: 2c
2024-07-25 11:57:20,494 - Using SNMP version: 2c
2024-07-25 11:57:20,697 - Setting the value of theRouter
2024-07-25 11:57:20,697 - Running updateNetworks
2024-07-25 11:57:20,697 - No current networks in Micetro matching router: M4100S
2024-07-25 11:57:20,697 - Looping through Interfaces:
2024-07-25 11:57:20,697 - Processing Interface: 66
2024-07-25 11:57:20,713 - Updating subnet 10.40.0.0/24
2024-07-25 11:57:20,713 - Updating Subnet Parameters {'discoveredRangeProperties': [{'rangeRef': 'ranges/2', 'discoveredProperties': [{'routerName': 'M4100S', 'gateway': '10.40.0.4', 'interfaceID': 66, 'interfaceName': 'vlan 1040'}, {'routerName': 'M4100S', 'gateway': '10.40.0.5', 'interfaceID': '66', 'interfaceName': 'vlan 1040', 'interfaceDescription': '', 'VRName': ''}]}]}
2024-07-25 11:57:20,728 - Processing Interface: 68
2024-07-25 11:57:20,728 - Updating subnet 10.50.0.0/24
2024-07-25 11:57:20,728 - Updating Subnet Parameters {'discoveredRangeProperties': [{'rangeRef': 'ranges/3', 'discoveredProperties': [{'routerName': 'M4100S', 'gateway': '10.50.0.4', 'interfaceID': 68, 'interfaceName': 'vlan 1050'}, {'routerName': 'M4100S', 'gateway': '10.50.0.5', 'interfaceID': '68', 'interfaceName': 'vlan 1050', 'interfaceDescription': '', 'VRName': ''}]}]}
2024-07-25 11:57:20,744 - Processing Interface: 67
2024-07-25 11:57:20,744 - Updating subnet 192.168.12.0/24
2024-07-25 11:57:20,744 - Updating Subnet Parameters {'discoveredRangeProperties': [{'rangeRef': 'ranges/4', 'discoveredProperties': [{'routerName': 'M4100S', 'gateway': '192.168.12.4', 'interfaceID': 67, 'interfaceName': 'vlan 1012', 'interfaceDescription': 'Internal', {'routerName': 'M4100S', 'gateway': '192.168.12.5', 'interfaceID': 67, 'interfaceName': 'vlan 1012', 'interfaceDescription': '', 'VRName': ''}]}]}
2024-07-25 11:57:20,760 - Processing Interfaces: 53
2024-07-25 11:57:20,760 - Updating subnet 192.168.88.0/24
2024-07-25 11:57:20,760 - Updating Subnet Parameters {'discoveredRangeProperties': [{'rangeRef': 'ranges/5', 'discoveredProperties': [{'routerName': 'M4100S', 'gateway': '192.168.88.4', 'interfaceID': 53, 'interfaceName': 'CPU Interface for Slot: 5 Port: 1', 'interfaceDescription': '', 'VRName': ''}]}]}
2024-07-25 11:57:20,775 - Network Updates Complete
2024-07-25 11:57:20,775 - Running updateRouterDevice
2024-07-25 11:57:20,838 - Running getARPEntriesWithSNMP
2024-07-25 11:57:20,838 - Getting ARP data from 10.50.0.5
2024-07-25 11:57:20,838 - Using SNMP version: 2c
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.1/68:30:e0:d8:c4:f3
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.3/00:30:18:08:02:c5
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.5/28:c6:8e:a1:6f:9f
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.13/7c:47:97:17:7c
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.14/0c:c4:7a:c8:7b:ce
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.40.0.146/5c:e9:1e:bc:34:ae
2024-07-25 11:57:21,056 - Discovered IP/MAC: 192.168.12.3/00:30:18:08:02:c4
2024-07-25 11:57:21,056 - Discovered IP/MAC: 192.168.12.5/28:c6:8e:a1:6f:9f
2024-07-25 11:57:21,056 - Discovered IP/MAC: 192.168.12.7/64:4b:f0:13:b6:aa
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.50.0.3/00:30:18:08:02:c6
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.50.0.5/28:c6:8e:a1:6f:9f
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.50.0.120/00:c2:b1:6f:0a
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.50.0.130/00:c2:29:28:bb:aa
2024-07-25 11:57:21,056 - Discovered IP/MAC: 10.50.0.140/00:c2:29:d9:4c:bd
2024-07-25 11:57:21,056 - Running updateARPEntries:
2024-07-25 11:57:21,056 - discoParameters {'discoveryInfoEntries': [{"addrRef": "#8-10.40.0.13", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "68:30:e0:d8:c4:f3", {"addrRef": "#8-10.40.0.3", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:30:18:08:02:c5", {"addrRef": "#8-10.40.0.5", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "28:c6:8e:a1:6f:9f", {"addrRef": "#8-10.40.0.13", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "7c:47:97:17:7c", {"addrRef": "#8-10.40.0.14", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "0c:c4:7a:c8:7b:ce", {"addrRef": "#8-10.40.0.146", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "5c:e9:1e:bc:34:ae", {"addrRef": "#8-192.168.12.3", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:30:18:08:02:c4", {"addrRef": "#8-192.168.12.5", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "28:c6:8e:a1:6f:9f", {"addrRef": "#8-192.168.12.7", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "64:4b:f0:13:b6:aa", {"addrRef": "#8-10.50.0.3", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:30:18:08:02:c6", {"addrRef": "#8-10.50.0.5", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "28:c6:8e:a1:6f:9f", {"addrRef": "#8-10.50.0.120", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:c2:b1:6f:0a", {"addrRef": "#8-10.50.0.130", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:c2:29:28:bb:aa", {"addrRef": "#8-10.50.0.140", "lastSeenDate": "2024-07-25 11:57:21", "lastDiscoveryDate": "2024-07-25 11:57:21", "clientIdentifier": "00:c2:d9:4c:bd"}]}]}
2024-07-25 11:57:21,088 - Processing Router Profile: vynos
```

Appendix

Common issues

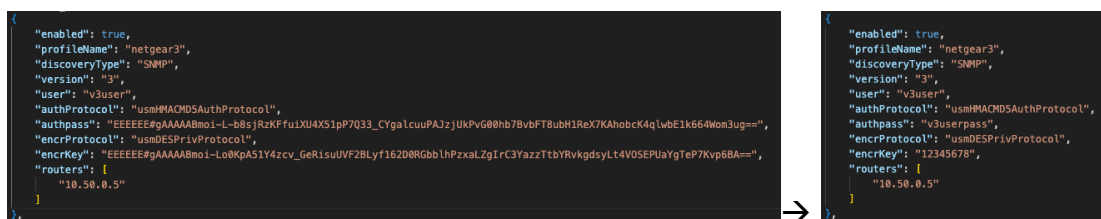
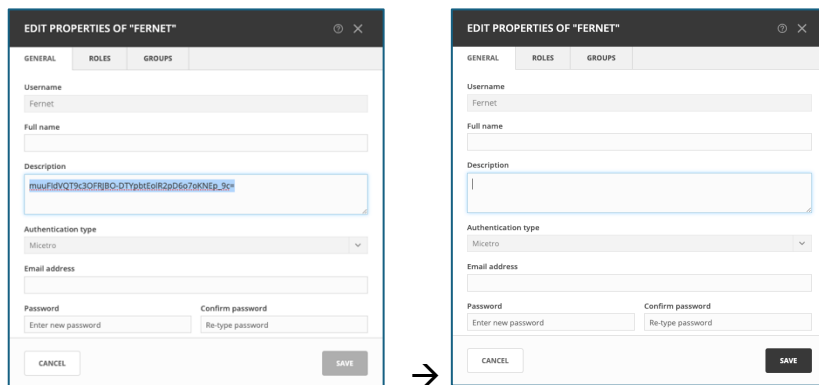
Encrypted credentials

If you see errors like the below in the logs it's likely that the SNMPv3 and API credentials stored in profiles.json were encrypted using a different Fernet key than the one stored in the Description Field of the fernetUser

```
2024-07-25 09:46:01,038 - authpass encrypted. unpacking using FernetKey
2024-07-25 09:46:01,038 - Failed to decrypt credential using Fernet. Was this encrypted using a
different Fernet key?
2024-07-25 09:46:01,038 - encrKey encrypted. unpacking using FernetKey
2024-07-25 09:46:01,038 - Failed to decrypt credential using Fernet. Was this encrypted using a
different Fernet key?
```

To work around this issue.

- Delete the Fernet key from the description field, and reset the credentials in profiles.json to clear text.



At the next script run a new Fernet key will be created and the credentials will be re-encrypted

Backing up the SQLite database

You may want to backup the database before running the externalDiscovery script. Useful for lab testing...

- Stop the central daemon/service
- Backup the current db
- Start central

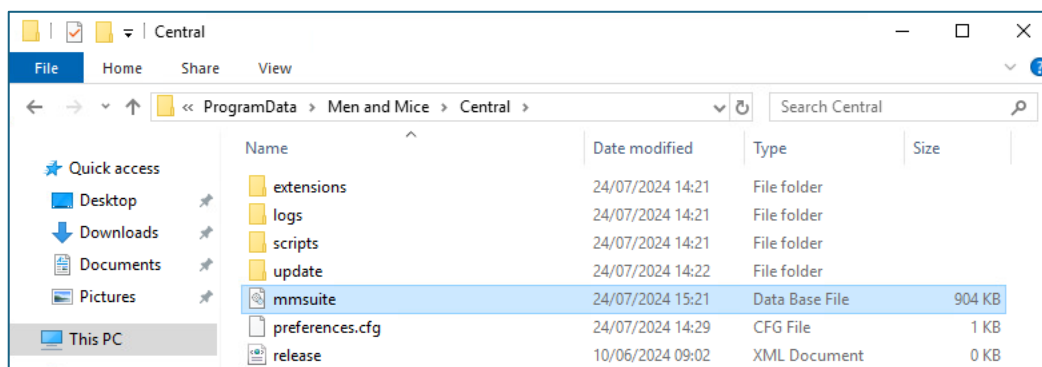
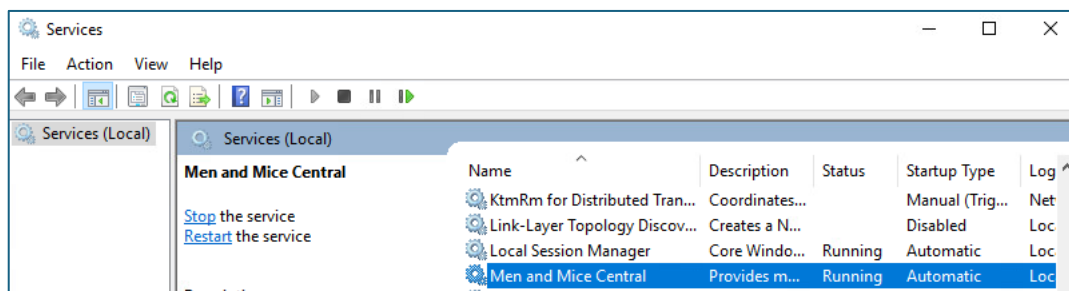
Linux

```
systemctl stop mmcentral
```

```
cd /var/mmsuite/mmcentral
cp mmsuite.db mmsuite.bak
```

```
systemctl start mmcentral
```

Windows



Using snmpwalk to test different oids

Using snmpwalk can be useful to try and identify alternative oids that could be used to collect the data useful for discovery. For example querying 1.3.6.1.2.1.4.35.1.4 against a Cisco Firepower gives similar results to 1.3.6.1.2.1.4.22.1.2 from the standard oids

Default SNMP OIDS used by Micetro

https://docs.menandmice.com/en/latest/guides/reference/snmp_oid/

IP-MIB:

ipNetToMediaPhysAddress: .1.3.6.1.2.1.4.22.1.2

ipAdEntAddr: .1.3.6.1.2.1.4.20.1.1

ipAdEntIfIndex: .1.3.6.1.2.1.4.20.1.2

ipAdEntNetMask: .1.3.6.1.2.1.4.20.1.3

ENTITY-MIB:

entPhysicalSerialNum: .1.3.6.1.2.1.47.1.1.1.1.11

SNMPv2-MIB:

sysName: .1.3.6.1.2.1.1.5

CISCO-VLAN-IFTABLE-RELATIONSHIP-MIB:

cvrRoutedVlanIfIndex: .1.3.6.1.4.1.9.9.128.1.1.1.1.3

CISCO-VRF-MIB:

cvVrflInterfaceType: .1.3.6.1.4.1.9.9.711.1.2.1.1.2

cvVrfName: .1.3.6.1.4.1.9.9.711.1.1.1.1.2

Testing using snmpwalk

snmpwalk -v 2c -c Community_string router_ip oid

Eg.

- SNMPv3 query for System Name

```
$ snmpwalk -v 3 -l authPriv -a SHA -u v3user -A "v3userpass" -x AES -X "12345678"
192.168.12.3 1.3.6.1.2.1.1.5
SNMPv2-MIB::sysName.0 = STRING: vrcore0
```

- SNMPv2 query for ARP tables

```
$ snmpwalk -v 2c -c routers 10.50.0.4 1.3.6.1.2.1.4.22.1.2
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.1 = STRING: 60:38:e0:d8:c4:f3
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.3 = STRING: 0:30:18:8:2:c5
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.4 = STRING: 6c:b0:ce:e6:e4:9c
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.13 = STRING: c:c4:7a:97:17:7c
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.14 = STRING: c:c4:7a:c8:78:ce
IP-MIB::ipNetToMediaPhysAddress.66.10.40.0.146 = STRING: 5c:e9:1e:bc:34:ae
IP-MIB::ipNetToMediaPhysAddress.67.192.168.12.3 = STRING: 0:30:18:8:2:c4
IP-MIB::ipNetToMediaPhysAddress.67.192.168.12.4 = STRING: 6c:b0:ce:e6:e4:9c
IP-MIB::ipNetToMediaPhysAddress.68.10.50.0.3 = STRING: 0:30:18:8:2:c6
IP-MIB::ipNetToMediaPhysAddress.68.10.50.0.4 = STRING: 6c:b0:ce:e6:e4:9c
IP-MIB::ipNetToMediaPhysAddress.68.10.50.0.120 = STRING: 0:c:29:b1:6f:a
IP-MIB::ipNetToMediaPhysAddress.68.10.50.0.130 = STRING: 0:c:29:28:bb:aa
```

FORTI-OS

The FORTIOS API doesn't appear to support retrieving the ARP tables from the device. In order to try and mitigate this we use the following to get associated MAC Addresses from DHCP and associated interfaces:

DHCP Database

api/v2/monitor/system/dhcp

Eg:

```
{
  "http_method": "GET",
  "results": [
    {
      "ip": "10.100.0.21",
      "reserved": false,
      "mac": "00:0c:29:99:9a:62",
      "vci": "udhcp 1.36.1",
      "hostname": "box",
      "expire_time": 1735573006,
      "status": "leased",
      "interface": "port2",
      "type": "ipv4",
      "server_mkey": 2,
      "server_ipam_enabled": false
    },
    {
      "ip": "10.100.0.22",
      "reserved": false,
      "mac": "00:0c:29:6d:d3:19",
      "vci": "udhcp 1.36.1",
      "hostname": "box",
      "expire_time": 1735573032,
```

```
    "status": "leased",
    "interface": "port2",
    "type": "ipv4",
    "server_mkey": 2,
    "server_ipam_enabled": false
  },
  {
    "ip": "10.100.0.20",
    "reserved": true,
    "mac": "00:0c:29:d7:99:20",
    "vci": "udhcp 1.36.1",
    "hostname": "box",
    "expire_time": 1735572806,
    "status": "leased",
    "interface": "port2",
    "type": "ipv4",
    "server_mkey": 2,
    "server_ipam_enabled": false
  },
  {
    "ip": "10.110.0.21",
    "reserved": false,
    "mac": "00:0c:29:6d:d3:23",
    "vci": "udhcp 1.36.1",
    "hostname": "box",
    "expire_time": 1735575968,
    "status": "leased",
    "interface": "port3",
    "type": "ipv4",
    "server_mkey": 3,
    "server_ipam_enabled": false
  },
  {
    "ip": "10.110.0.20",
    "reserved": false,
    "mac": "00:0c:29:99:9a:6c",
    "vci": "udhcp 1.36.1",
    "hostname": "box",
    "expire_time": 1735576032,
    "status": "leased",
    "interface": "port3",
    "type": "ipv4",
    "server_mkey": 3,
    "server_ipam_enabled": false
  }
],
"vdom": "root",
"path": "system",
"name": "dhcp",
"status": "success",
"serial": "FGVMEV6HP7EHLVCB",
"version": "v7.4.3",
"build": 2573
}
```

Interfaces

/api/v2/monitor/system/interface

```
{
  "http_method": "GET",
  "revision": "1735572564.481479",
  "results": {
    "port1": {
      "id": "port1",
      "name": "port1",
      "alias": "(port1-alias)",
      "mac": "00:0c:29:bc:99:1a",
      "ip": "10.50.0.250",
      "mask": 24,
      "link": true,
      "speed": 10000.0,
      "duplex": 1,
      "tx_packets": 1748616,
      "rx_packets": 3966682,
      "tx_bytes": 282896925,
      "rx_bytes": 1188365152,
      "tx_errors": 0,
      "rx_errors": 0
    },
    "port2": {
      "id": "port2",
      "name": "port2",
      "alias": "",
      "mac": "00:0c:29:bc:99:24",
      "ip": "10.100.0.250",
      "mask": 24,
      "link": true,
      "speed": 10000.0,
      "duplex": 1,
      "tx_packets": 382992,
      "rx_packets": 1857258,
      "tx_bytes": 21558766,
      "rx_bytes": 531914803,
      "tx_errors": 0,
      "rx_errors": 0
    },
    "port3": {
      "id": "port3",
      "name": "port3",
      "alias": "",
      "mac": "00:0c:29:bc:99:2e",
      "ip": "10.110.0.250",
      "mask": 24,
      "link": true,
      "speed": 10000.0,
      "duplex": 1,
      "tx_packets": 243480,
      "rx_packets": 223548,
      "tx_bytes": 13994197,
      "rx_bytes": 15310004,
      "tx_errors": 0,
      "rx_errors": 0
    }
  }
}
```

```

    }
  },
  "vdom": "root",
  "path": "system",
  "name": "interface",
  "status": "success",
  "serial": "FGVMEV6HP7EHLVCB",
  "version": "v7.4.3",
  "build": 2573
}

```

Creating binary executables from python code using pyinstaller

Pyinstaller <https://pyinstaller.org/en/stable/> can be used to generate Linux and Windows native executables. The advantages of this are –

- No need for the customer/prospect to install python and download dependencies
- Source code is hidden

Before building a native executable python first needs to be installed in a matching OS environment together with all dependant python modules

externalDiscovery requires the following python packages not included in the standard install

- requests
- json
- ipaddress
- logging
- cryptography

pyinstaller itself is written in python and can also be installed using pip in the standard way

Creating the binary

Binaries were created using the following-

pyinstaller -F externalDiscovery.py

The -F option creates a one file bundled executable

API Commands used to retrieve data

Cisco Nexus

<https://developer.cisco.com/docs/nx-api-cli-reference-for-the-cisco-nexus-7000-series-platform/overview/>

The API can be used to expose the CLI

Get hostname	Show hostname
Get Interfaces (incl. vrf info)	show ip interface vrf all
Get ARP entries (incl vrf info)	show ip arp detail vrf all

Palo Alto

<https://docs.paloaltonetworks.com/pan-os/10-1/pan-os-panorama-api/get-started-with-the-pan-os-rest-api/pan-os-rest-api>

Operational commands accessed by `/api/?type=op&cmd=`

Get the hostname	<code><show><system><info></info></system></show></code>
Get Interfaces	<code><show><interface>all</interface></show></code>
Get the management interface (not incl. in get interfaces)	<code><show><interface>management</interface></show></code>
Get ARP entries	<code><show><arp><entry name = 'all'/></arp></show></code>

Configuration commands accessed by `/api/?type=config&action=get&xpath=`

Get NAT rules	<code>/config/devices/entry[@name='localhost.localdomain']/vsys/entry[@name='vsys1']/rulebase/nat</code>
---------------	--

Generate a key

https://firewall_ip/api/?type=keygen&user=username&password=password

Create a fake API for testing

If you have some sample output data, perhaps provided by the Customer you can simply create a fake API interface using python.

For example -

Create a sub-directory named api (target for PaloAlto API calls)

Create index.html and paste in the XML data provided by the customer. Eg.

```
<response status="success">
<result>
<nat>
<rules>
<entry name="MB-Test" uuid="dbd38cc3-2fa4-4958-bf86-71fef07dfe5c">
<to>
<member>External</member>
</to>
<from>
<member>Internal</member>
</from>
<source>
<member>any</member>
</source>
<destination>
<member>dummy external</member>
</destination>
<service>any</service>
<description>Testing</description>
<source-translation>
<dynamic-ip-and-port>
<translated-address>
<member>195.200.201.99</member>
</translated-address>
</dynamic-ip-and-port>
</source-translation>
<destination-translation>
<translated-address>195.200.201.99</translated-address>
</destination-translation>
</entry>
</rules>
</nat>
```



```
</result>  
</response>
```

Use python to start a webserver listening on localhost / port 8000

```
$ python3.10 -m http.server -b 127.0.0.1 8000
```

You should now be able to parse the HTTP from the script for testing (localhost:8000)

Obviously this doesn't simulate authentication etc.. But is still useful if we don't have the type of device the customer is using available to test against.